BrewStack

A centralised platform for Singapore's specialty coffee ecosystem. BrewStack aggregates bean offerings from local roasters into a single, daily updated catalogue, helping consumers discover new coffees and giving smaller roasters more visibility.

Currently implemented features:

- Web scraping for Homeground, Nylon, and Tiong Hoe
- Central catalog page with live scraped data

Milestone 1 Links

Project Log: https://docs.google.com/spreadsheets/d/1qJc9qv75R07gXsJ0Ww4iWmQ5QGgNuagz5SHjS-Yqv0I/edit?usp=sharing

Project Poster: https://drive.google.com/file/d/1z_YSlxhBxBeHbIyGiQYEOPobfASUg1s2/view?usp=sharing

Project Video: https://drive.google.com/file/d/1-EOzyGoWDlaDIFwZOm83wMpufqB47dG-/view?usp=sharing

---

Tech Stack

Layer	Technology
Frontend	React + TypeScript (Vite)
Backend	Express.js + TypeScript
Database	PostgreSQL (Supabase)
ORM	Prisma
Scraping	Playwright
Scheduling	node-cron
Styling	CSS

Project Structure

```
BrewStack/
├── client/                  # React frontend
│   └── src/
│       ├── pages/
│       │   ├── Home.tsx
│       │   ├── Catalog.tsx
│       │   └── BeanDetail.tsx
```

```
│       ├── components/
│       └── types/
├── src/                    # Express backend
│   ├── db/
│   │   ├── client.ts       # Prisma client singleton
│   │   └── upsert.ts       # Bean + roaster upsert logic
│   ├── routes/             # Express API routes
│   └── scraper/
│       ├── scrapers/
│       │   ├── BaseScraper.ts
│       │   ├── HomegroundScraper.ts
│       │   ├── NylonScraper.ts
│       │   └── TiongHoeScraper.ts
│       ├── types/
│       ├── utils/
│       │   ├── browser.ts
│       │   ├── parse.ts
│       │   └── normalise.ts
│       └── scheduler.ts    # Cron job (nightly, 2 AM SGT)
├── prisma/
│   └── schema.prisma
└── index.ts                # Express entry point
```

Getting Started

Prerequisites

- Node.js v18+
- A Supabase project (for the PostgreSQL database)

Installation

```
# Clone the repo
git clone https://github.com/edwardheww/BrewStack.git
cd BrewStack

# Install backend dependencies
npm install

# Install Playwright browser
npx playwright install chromium

# Install frontend dependencies
cd client && npm install && cd ..
```

Environment Variables

Create a `.env` file at the project root:

```
DATABASE_URL=your_supabase_connection_string
PORT=3000
```

Database Setup

```
npx prisma migrate dev
```

Running the App

```
# Terminal 1 — backend
npm run dev

# Terminal 2 — frontend
cd client && npm run dev
```

The frontend runs at `http://localhost:5173` and the backend at `http://localhost:3000`. As of Milestone 1, only `http://localhost:5173/catalog` has content.

How It Works

Scraper

The scraper is built around an abstract `BaseScraper` class. Every roaster-specific scraper extends it and implements a single `scrape()` method. The base class handles the shared logic: launching a Playwright browser, navigating to the roaster's catalog page, wrapping the scrape in error handling, and returning a structured `ScrapeResult`.

Each roaster scraper works in two passes:

1. **Catalog pass** — visits the roaster's bean listing page and collects all product URLs (or handles, for Shopify stores)
2. **Detail pass** — visits each product page individually and extracts fields like name, varietal, processing method, roast level, tasting notes, and image URL

Field extraction varies per roaster since each site has a different HTML structure. Selectors are tuned per scraper using browser DevTools.

All scrapers are registered in `scheduler.ts`. A nightly cron job at 2 AM SGT runs them sequentially with a short random delay between each roaster to avoid triggering bot detection. After scraping, results are passed to `upsertScrapedBeans()`.

Supported roasters:

- Homeground Coffee Roasters
- Nylon Coffee Roasters
- Tiong Hoe Specialty Coffee

Database

Two Prisma models are used:

Roaster — stores the roaster's name and website. Uniquely identified by name.

Bean — stores all scraped bean data. Uniquely identified by URL, so re-running the scraper updates existing records rather than creating duplicates. Optional fields (varietal, roast level, tasting notes, etc.) account for roasters that don't publish all metadata. Each bean has a foreign key back to

its roaster.

On each scrape run, the database is cleared for that roaster before upserting fresh data. This ensures beans that have been removed from a roaster's site don't persist in the catalogue.

API

The Express backend exposes two REST endpoints:

- `GET /beans` — returns all beans, each with its nested roaster object included
- `GET /roasters` — returns all roasters

Frontend Catalog

The catalog page fetches from `GET /beans` on load using `useEffect`, storing the response in React state. Beans with missing tasting notes are filtered out client-side before rendering. Each bean is displayed as a card showing the name, roaster, tasting notes, roast level, processing method, and price.

Adding a New Roaster

1. Duplicate any existing scraper in `src/scraper/scrapers/`
2. Update the `Roaster` config (name, website, catalog URL)
3. Inspect the roaster's site in DevTools and update the CSS selectors
4. Register the new scraper in `src/scraper/scheduler.ts`

Data Model

The full Prisma schema is as follows:

```
model Roaster {
  id      String @id @default(cuid())
  name    String @unique
  website String

  beans Bean[]
}

model Bean {
  id               String   @id @default(cuid())
  name             String
  price            Float?
  url              String?  @unique
  imageUrl         String?
  roastLevel       String?
  varietal         String?
  flavourNotes     String?
  processingMethod String?
  updatedAt        DateTime @updatedAt

  roasterId String
  roaster   Roaster @relation(fields: [roasterId], references: [id])
}
```

Design decisions:

`id` is auto-generated by Prisma using `cuid()` for both models, so neither the scraper nor the API needs to manage primary keys manually.

`name` on `Roaster` is marked `@unique` since two roasters will never share the same name. This is used as the match key when upserting roasters — if a roaster already exists, its website is updated; otherwise a new record is created.

`url` on `Bean` is marked `@unique` and is the primary match key for upserts. Since each bean's product page has a stable URL, this reliably identifies whether a bean already exists in the database across scrape runs. It is nullable because some scrapers may not always produce a URL, in which case the bean is still inserted but cannot be updated on subsequent runs.

Most `Bean` fields are nullable (`?`) because different roasters publish different levels of detail. Homeground, for example, lists varietal, processing method, and tasting notes consistently, while other roasters may omit some of these. Making them optional prevents scrape failures when a field is simply absent on a page.

`updatedAt` is managed automatically by Prisma and reflects the last time a given bean record was written to. This is useful for debugging stale data and will eventually support "freshness" indicators on the frontend.

The `roasterId` foreign key links each bean back to its roaster, and the `roaster` relation allows Prisma to return the full roaster object alongside each bean in a single query using `include: { roaster: true }`.

Challenges & Solutions

Bot Detection on Shopify Stores

Several roasters (Homeground, Nylon) use Shopify, which detects and blocks headless browsers through fingerprinting and behavioural analysis. This caused scrapers to time out or receive empty pages mid-run.

We addressed this by launching Playwright with `slowMo` to introduce delays between browser actions, making the scraper behave more like a human user. We also run scrapers sequentially rather than in parallel, with an additional random delay between roasters, to avoid hammering any single server. Where Shopify's `.json` product endpoints are available, we use those instead of scraping rendered HTML.

Inconsistent HTML Structures Across Roasters

Each roaster's website is built differently — some use Shopify's custom `<product-card>` elements, others use standard HTML with varying class naming conventions. There is no universal selector that works across all sites.

We solved this by giving each roaster its own scraper class with selectors tuned specifically to that site. The `BaseScraper` abstract class enforces a consistent interface (`scrape()` returning

`Bean[]`) while leaving the implementation entirely up to each subclass. This makes it easy to add new roasters without touching existing scrapers.

Missing or Inconsistently Structured Data

Tasting notes, roast level, and other fields are not always present or consistently formatted. On some pages, notes are inside `<em>` tags; on others they are in paragraph elements with specific class names. Prices may or may not be listed. Some beans appear on a catalog page but have sparse detail pages.

We handled this by making all non-essential fields optional on the `Bean` model, wrapping individual field extractions in fallback expressions (`?? ''`), and wrapping each product page scrape in a `try/catch` so a single failing product does not abort the entire roaster's run. Failed URLs are logged to the console for manual inspection.

Keeping Data Fresh Without Duplication

Running the scraper nightly risks accumulating stale records if a roaster removes a bean from their catalogue — the old record would persist indefinitely without a removal mechanism.

We addressed this by deleting all of a roaster's beans from the database before each scrape run, then reinserting fresh data. This guarantees the database always reflects the current state of a roaster's catalogue. The trade-off is that historical data is not retained, which is acceptable for Milestone 1 but may be revisited in later milestones.

Planned Features

Milestone 2

Coffee Discovery Chatbot A conversational interface that helps users narrow down beans based on their taste preferences. Users describe what they enjoy — fruity, floral, chocolatey, light roast, natural process — and the chatbot surfaces matching beans from the catalogue. Planned to be implemented using an LLM API with the bean database as context.

Interactive Roaster Map A map view built with Mapbox showing all supported roasters as pins across Singapore. Users can click a pin to see the roaster's current bean offerings, opening a filtered view of the catalogue. Useful for consumers who want to discover roasters near them or plan a coffee crawl.

Bean Detail Page A dedicated page for each bean showing its full profile — origin, farm, varietal, processing method, roast level, tasting notes, and price — alongside a link to purchase directly from the roaster's website.

Milestone 3

User Accounts & Saved Beans Users can create accounts and save beans they are interested in or have tried. Saved beans persist across sessions and can be used to generate personalised recommendations.

Community Reviews & Ratings Authenticated users can leave short reviews and star ratings on individual beans. Aggregate ratings are displayed on bean cards in the catalogue, helping newer users make informed choices based on community feedback.

Direct Purchase Integration Integration with roasters' existing checkout flows, allowing users to add beans to a cart and complete a purchase without leaving BrewStack.

API Reference

Method	Endpoint	Description
GET	`/beans`	Returns all beans with roaster info
GET	`/roasters`	Returns all roasters

Made By

- Edward Hew
- Lim Jun Hong